

《算法设计技巧与分析》

第12章 回溯法

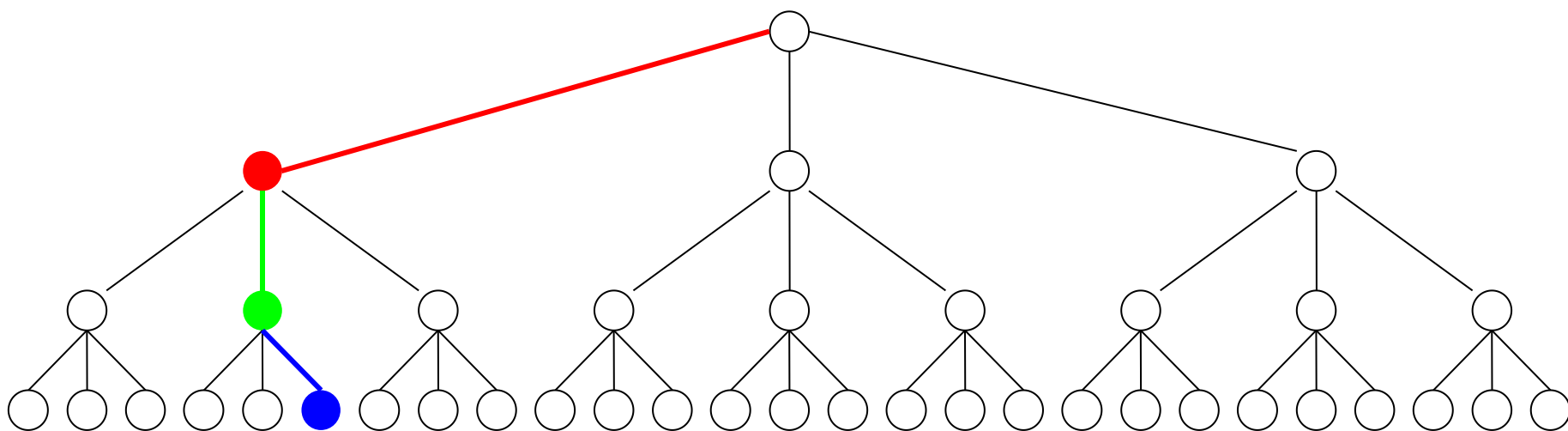
朱友文

zhuyw@nuaa.edu.cn

引例

- 图的3着色问题: **NP完全问题**
- 给定一个无向图 $G=(V,E)$, 及3种颜色 $\{1,2,3\}$, 现要为图的每个顶点着色。每顶点只能着一种颜色, 并且要求相邻的顶点具有不同的颜色
- 一个具有 n 个顶点的图, 可用一个 n 维的向量 $(c_1, c_2, \dots, c_n)$ 表示一种着色方案, $c_i \in \{1,2,3\}$, $i=1,2,\dots,n$, 共有 3^n 种可能的着色, 可用一棵完全的3叉树表示

- 下图为有3个顶点的所有可能的着色搜索树，从根到叶节点的每条路径表示一种着色方案



(c_1, c_2, c_3)

当只有3个节点的时候 (即 $n=3$)

12.1 引言: 回溯法的工作原理

- 回溯法的基本做法是搜索，是一种组织得井井有条，能避免不必要搜索的穷举式搜索法
- 回溯法按**深度优先策略**搜索问题的解空间树。算法搜索至解空间树的任意一节点时，先判断该节点是否可能包含问题的解：
 - ✓ 1) 如果**肯定不包含**，则跳过这个节点及其子孙节点；
 - ✓ 2) 如果**可能包含**，进入该子树，继续按深度优先策略搜索；
 - ✓ 3) 若某节点 i 的**所有子节点都不可能包含**问题的解，则回溯到 i 的父节点，继续搜索父节点和下一个孩子节点

12.1 引言

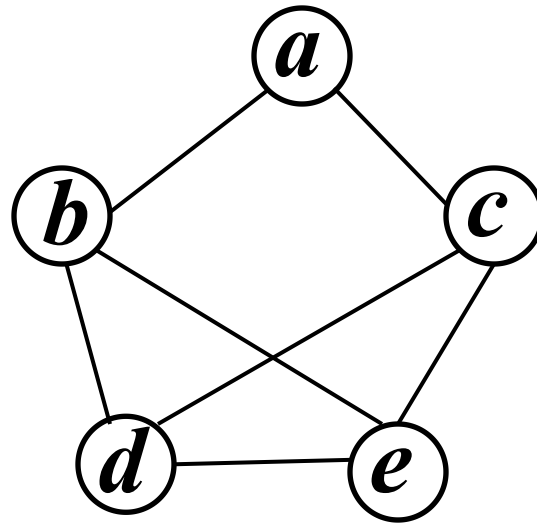
➤ 回溯法：有组织的穷尽搜索

- ✓ 有明确的适用对象

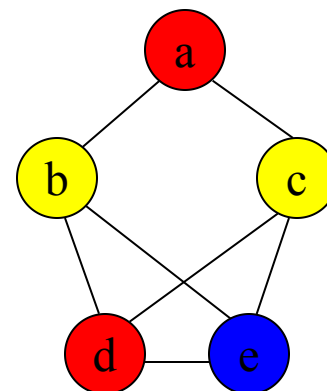
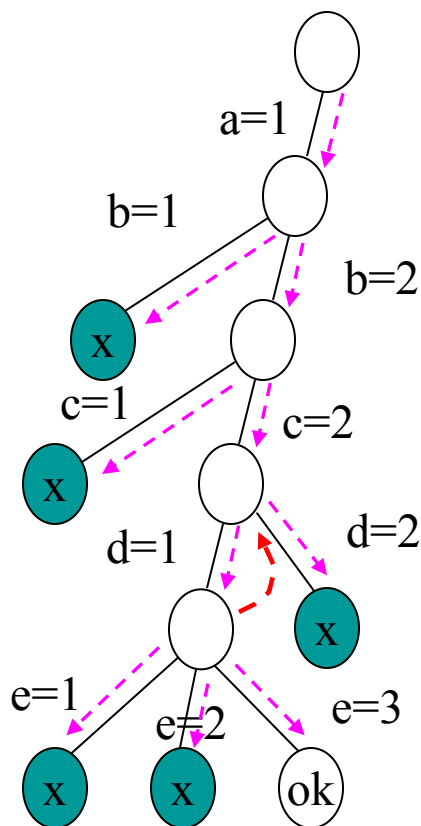
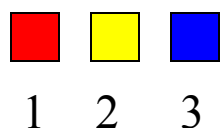
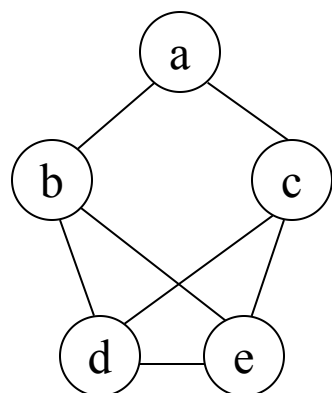
- ✓ 有明确的设计步骤

例子： 3着色问题

➤ 3着色问题： 无向图 $G=(V, E)$



例12.1



“合法着色” – 全部顶点都已经被着色，且没有两个相邻的顶点是同样的颜色。

“部分着色” – 部分顶点还未着色；在已经着色的顶点中，没有两个相邻的顶点是同样的颜色。

12.2 3着色问题

➤ 3着色问题：无向图 $G=(V, E)$

$$V=\{v_1, v_2, \dots, v_n\}$$

$$c_i=\{1,2,3\} \quad // \text{颜色集合}$$

$$\text{每种着色方法: } (c_1, c_2, \dots, c_n)$$

12.2 3着色问题

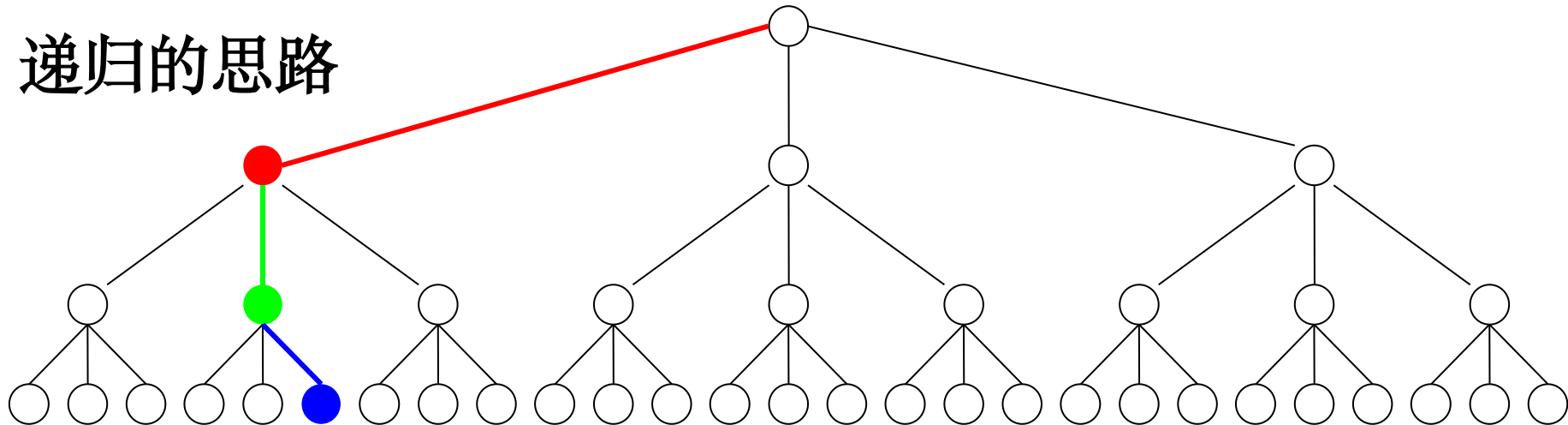
➤ 3着色问题：无向图 $G=(V, E)$

$$V=\{v_1, v_2, \dots, v_n\}$$

$$c_i=\{1,2,3\} \quad // \text{颜色集合}$$

每种着色方法： $(c_1, c_2, \dots, c_n)$

递归的思路



12.2 3着色问题

➤ 3着色问题：无向图 $G=(V, E)$

递归算法：主程序

算法 12.1 3-COLORREC

输入：无向图 $G=(V, E)$ 。

输出： G 的顶点的3着色 $c[1....n]$ ，其中每个 $c[j]$ 为1,2或3。

for ($k \leftarrow 1$ to n)

$c[k] \leftarrow 0$;

end for

flag $\leftarrow$ false;

graphcolor(1);

if (flag) then output c ;

else output “no solution”;

graphcolor(i) 表示对节点 i 到 n 进行着色

12.2 3着色问题

➤ 3着色问题：无向图 $G=(V, E)$

递归算法：递归子程序

```
graphcolor(k)
{ for(color = 1 to 3){
    c[k]←color;
    if (c 为合法着色) flag ← true and exit;
    else if (c 是部分合法的) graphcolor(k+1);
}
}
```

12.2 3着色问题

➤ 3着色问题：无向图 $G=(V, E)$

迭代算法：框架

```
for (k ← 1 to n)  
    c[k] ← 0;
```

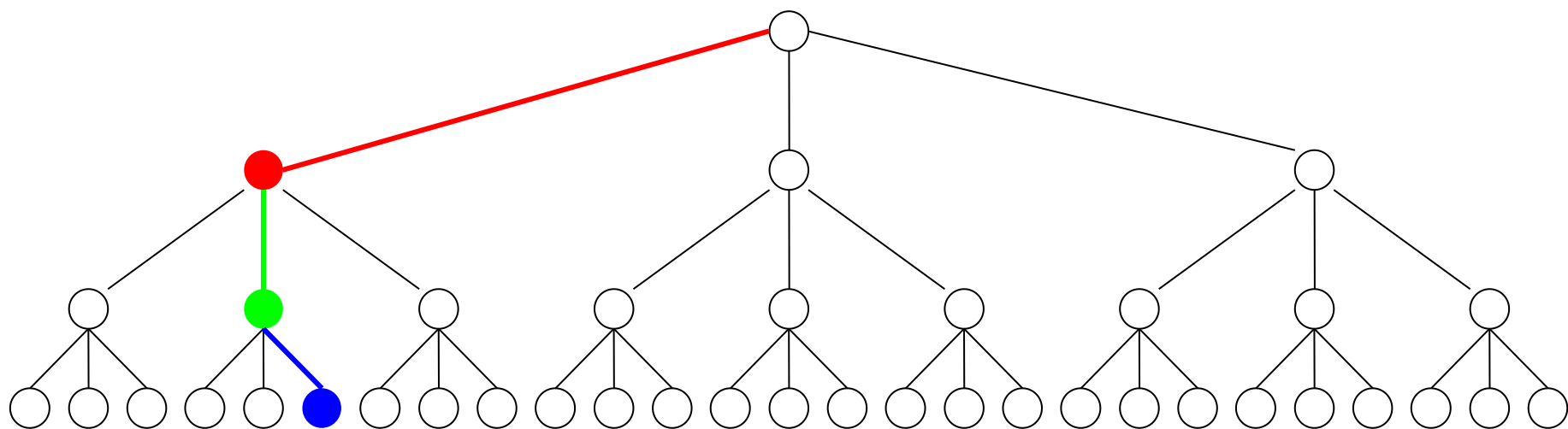
```
end for
```

```
flag ← false;
```

中间的迭代过程怎么设计？

```
if (flag) then output c;  
else output “no solution”;
```

- 观察一下遍历顺序

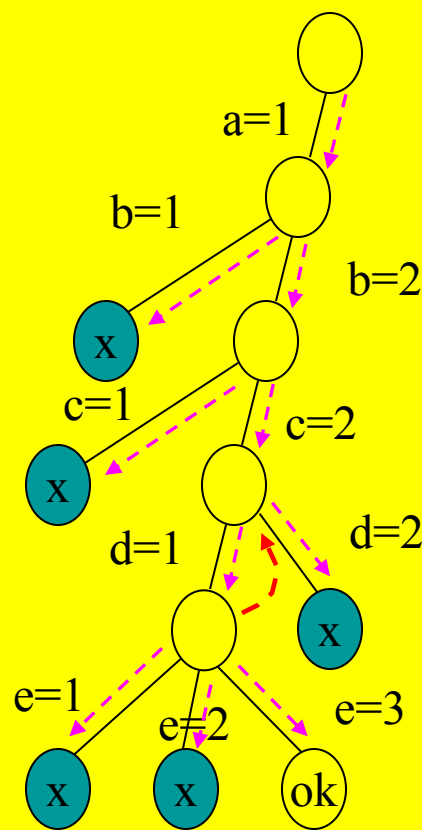


12.2 3着色问题

➤ 3着色问题：无向图 $G=(V, E)$

迭代算法：迭代过程

```
k=1;  
while (k≥1){  
  while (c[k]≤2) { //为第k个顶点着色  
    c[k] ← c[k]+1;  
    if (c为合法着色) flag ← true and 退出两重while  
    else if (c为部分着色) k ← k+1 //准备为下一顶点着色, →7  
  } //endwhile 所有着色都会出现冲突, 其父节点试其他色  
  c[k] ← 0 //vk试验了所有颜色均失败, 当前顶点颜色只好归0, 回溯  
  k ← k-1 //回溯到上一个顶点 (配合第6行的k≥1来理解)  
} //endwhile
```



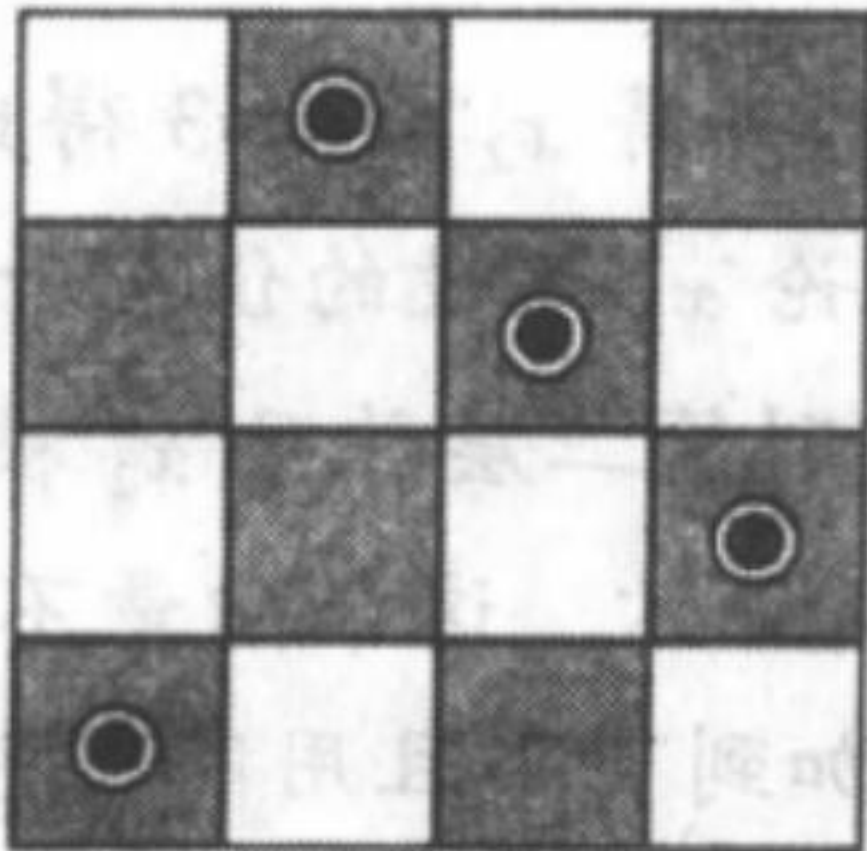
12.2 3着色问题

➤ 3着色问题：无向图 $G=(V, E)$

复杂度分析

12.3 n 皇后问题

➤ n 皇后问题：当 $n=4$ 时，



12.3 8皇后问题

➤ n 皇后问题：当 $n=4$ 时，

解的表示形式： $\{x_1, x_2, x_3, x_4\}$

互相攻击的情况：

(1) $x_i = x_j$

(2) $x_i - x_j = i - j$ 或者 $x_i - x_j = j - i$

12.3 8皇后问题

➤ n 皇后问题：当 $n=4$ 时，

递归算法： $c[1,...,4]$ 表示存放位置

主程序：

```
for( $k=1$  to  $4$ )  $c[k]=0$ ;
```

```
flag=false;
```

```
Rec4-queens( $1$ ); //递归子程序
```

```
if(flag=true) output  $c[]$ ;
```

```
else output “no solution”;
```

12.3 8皇后问题

➤ n 皇后问题：当 $n=4$ 时，

递归子程序：

Rec4-queens(k) { //递归子程序

for($col=1$ to 4){

$c[k]=col$;

if (c 全合法) { $flag=true$; $exit$;} }

else{ **if**(c 部分合法) **Rec4-queens**($k+1$); }

 }

}

12.3 8皇后问题

➤ n 皇后问题：当 $n=4$ 时，

迭代递归算法： $c[1,...,4]$ 表示存放位置

主程序：

```
for( $k=1$  to  $4$ )  $c[k]=0$ ;
```

```
flag=false;
```

```
//迭代过程
```

```
if(flag=true) output  $c[]$ ;
```

```
else output “no solution”;
```

12.3 8皇后问题

➤ n 皇后问题：当 $n=4$ 时，

迭代递归算法： $c[1,...,4]$ 表示存放位置

//迭代过程：

```
 $k=1$ ;  
while  $k \geq 1$   
    while  $c[k] \leq 3$  //为第 $k$ 个顶点位置  
         $c[k] = c[k] + 1$ ;  
        if ( $c$ 为合法摆放) then { $flag = true$  and 退出两重while}  
        else {if ( $c$ 为部分摆放) then  $k = k + 1$ ;} //准备放置下一顶点  
    end while //假设放置既不合法也非部分的，即死节点，试其他色  
     $c[k] = 0$ ; //  $c_k$ 试验了所有位置均失败，当前顶点位置只好归0，回溯  
     $k = k - 1$ ; //回溯到上一个顶点 (配合while的条件 $k \geq 1$ 来理解)  
end while //注意：回溯到上一个顶点后， $c[k] + 1$ ，即尝试下一种颜色
```

12.3 8皇后问题

➤ n 皇后问题:

简要分析:

蛮力法: $O(n^n)$, 或者 $O(n!)$

回溯法: 经验上的期望运行时间少

12.4 一般回溯法

➤ 回溯法适用于解决什么样的问题？

解的形式是定长向量

用数组 $c[1, \dots, n]$ 的形式表示,

12.4 一般回溯法

- 假设 $c[k]$ 的取值范围是集合 X_k ,
- 特殊情况 $X_1 = X_2 = \dots = X_n = \{1, 2, \dots, m\}$
- 两个基本的子步骤:
 - 判断 $c[1, \dots, n]$ 是否全合法
 - 判断 $c[1, \dots, k]$ 是否合法 (部分合法, $k < n$)

12.4 一般回溯法

➤ 回溯法的算法设计方法?

算法框架

```
初始化  $c[1, \dots, n]$ ;  
 $flag = false$ ;  
回溯过程: 递归/迭代;  
if ( $flag$ ) output  $c$ ;  
else output “no solution”;
```

12.4 一般回溯法

➤ 回溯法的算法设计方法?

递归回溯: 调用方法 `advance(1);`

```
advance(k){  
    for(每个  $x \in X_k$ ) { // ( $i=1; i \leq |X_k|; i++$ )  
         $c[k] = x$ ; //  $c[k] = X_k[i]$ ;  
        if(c合法) {  $flag=true, exit$ ; }  
        else { if(部分合法) advance( $k+1$ ); }  
    }
```

12.4 一般回溯法

➤ 回溯法的算法设计方法?

迭代回溯:

```
 $k=1$ ;  
while ( $k \geq 1$ )  
    while ( $X_k$ 尚未被穷举)  
         $c[k] = X_k$ 的下一个元素;  
        if ( $c$ 全合法) then { $flag = true$  and 退出两重while;}  
        else {if ( $c$ 为部分着色)then  $k = k + 1$ ;} //准备放置下一顶点  
    end while //假设放置既不合法也非部分的, 即死节点, 试其他  
    重置 $c[k]$ ; //  $c[k]$ 试验了所有可能均失败, 重置, 回溯  
     $k = k - 1$ ; //回溯到上一个顶点 (配合while的条件 $k \geq 1$ 来理解)  
end while //注意: 回溯到上一个顶点后,  $c[k] + 1$ , 即尝试下一种元素
```

12.4 一般回溯法

➤ 特殊情况 $X_1 = X_2 = \dots = X_n = \{1, 2, \dots, m\}$

➤ 算法框架

```
for(i=1; i ≤ n; i++) c[i]=0; //初始化
```

```
flag = false;
```

```
回溯过程：递归/迭代；
```

```
if (flag) output c;
```

```
else output “no solution”;
```

12.4 一般回溯法

➤ 特殊情况 $X_1 = X_2 = \dots = X_n = \{1, 2, \dots, m\}$

递归回溯: 调用方法 `advance(1);`

```
advance(k){  
    for( $x=1; x \leq m; x++$ )  
         $c[k] = x;$   
        if(c合法) { $flag=true, exit;$ }  
        else {if(部分合法)  $advance(k+1);$ }  
}
```

12.4 一般回溯法

➤ 特殊情况 $X_1 = X_2 = \dots = X_n = \{1, 2, \dots, m\}$

迭代回溯:

```
k=1;
while (k≥1){
    while (c[k] < m){
        c[k] = c[k] + 1; //Xk 的下一个元素
        if (c全合法) then {flag=true and 退出两重while;}
        else {if (c为部分合法) then k=k+1;} //准备放置下一顶点
    }end while //假设放置既不合法也非部分的, 即死节点, 试其他
    c[k] = 0; //c[k]试验了所有可能均失败, 重置, 回溯
    k = k-1; //回溯到上一个顶点 (配合while的条件k≥1来理解)
}end while //注意:回溯到上一个顶点后, c[k]+1, 即尝试下一种元素
```

12.4 一般回溯法

➤ 回溯法适用于解决什么样的问题？

解的形式是定长向量

如果不满足，怎么办？

12.4 一般回溯法

例12.3 考虑定义如下的PARTITION问题中的一个变型。给定一个 n 个整数的集合 $X=\{x_1, x_2, \dots, x_n\}$ 和整数 s ，找出和等于 s 的 X 的子集 P 。比如说，如果

$$X = \{10, 20, 30, 40, 50, 60\}$$

和 $s=60$ ，则有三种不同长度的解，它们分别是

$$\{10, 20, 30\}, \{20, 40\} \text{ 和 } \{60\}$$

12.4 一般回溯法

例12.3 考虑定义如下的PARTITION问题中的一个变型。给定一个 n 个整数的集合 $X=\{x_1, x_2, \dots, x_n\}$ 和整数 s ，找出和等于 s 的 X 的子集 P 。比如说，如果

$$X = \{10, 20, 30, 40, 50, 60\}$$

和 $s=60$ ，则有三种不同长度的解，它们分别是

$$\{10, 20, 30\}, \{20, 40\} \text{ 和 } \{60\}$$

将其解换一种表示方式，表示为定长的向量，比如该问题中，表示为6维0/1向量，

1表示包含该数据，0表示不包含

那么三组解对应的表示方式分别是 $(1, 1, 1, 0, 0, 0)$, $(0, 1, 0, 1, 0, 0)$, $(0, 0, 0, 0, 0, 1)$,

12.4 一般回溯法

如果求出所有的可行解，怎么做？